SIMATS ENGINEERING
ASSESSMENT TOOL 2

COURSE NAME: Artificial Intelligence **COURSE CODE: CSA17**

COURSE OUTCOME COVERED

CO1: Apply AI basics such as intelligent agents and uninformed search methods to solve problems effectively *(BL3)*

Assessment Tool Weightage: 50%

QUESTIONS: 3 **Duration : 1 Hour**
Total Marks:30

Annexure A

Constraint-Based Problem Solving

Code of Conduct:

I R.Nivetha /1912260554 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Nivetha

Annexure A

Constraint-Based Problem Solving

1. A hospital needs to assign 3 doctors (D1, D2, D3) to 3 shifts (Morning, Afternoon, Night) such that:

   Constraints:

   i. D1 cannot work Night shift
   ii. D2 must work before D3 (Morning < Afternoon < Night)
   iii. Only one doctor per shift
   iv. D3 cannot work Morning
   v. All doctors must be assigned exactly one shift

   Apply Backtracking Search to find a valid assignment with all intermediate steps and constraint checks. State the final valid schedule

2. A robot operates in a grid environment (5×5). It must move from Start (S) to Goal (G). Obstacles block certain paths.

   Constraints:

   i. Movement allowed: Up, Down, Left, Right
   ii. Cost of each move = 1
   iii. Cannot pass through obstacles
   iv. Use Manhattan Distance heuristic

   Using above constraints and BFS Search Algorithm Show step-by-step node expansion and priority queue updates and determine the optimal path and cost.

3. An autonomous rescue robot is deployed in a disaster-affected building to locate and rescue survivors. The robot operates in a partially observable environment where some paths are blocked, and it must make decisions with limited information.

The building is represented as a grid of rooms. The robot starts at position S and must reach a survivor located at G.

Constraints:

- The robot can move up, down, left, right
- Some cells are blocked (obstacles) and cannot be traversed
- The robot has limited sensing capability (can only observe adjacent cells)
- Each movement has a cost of 1, but entering risky zones has an additional cost of 2
- The robot must minimize total cost while ensuring safe navigation
- The robot must update its knowledge dynamically (online search scenario)

Tasks:

a) Analyze all the given constraints and explain how they affect the robot's decision-making process.

b) Develop a solution approach using an appropriate search strategy (e.g., Uniform Cost

Search / Online Search Agent). Clearly explain the steps involved.
c) Demonstrate how the robot applies AI concepts such as:

- Intelligent agents
- Rationality
- Environment type

d) Justify why your chosen approach is the most suitable for solving this problem under the given constraints.

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Understanding	25%	Clearly defines the problem and identifies all relevant constraints accurately	Identifies most constraints with minor gaps	Partial understanding; misses key constraints	Fails to identify constraints correctly
Constraint Analysis	25%	Analyzes constraints effectively and prioritizes them logically	Provides reasonable analysis with some prioritization	Limited analysis; weak prioritization	No meaningful analysis or prioritization
Solution Development	25%	Develops optimal and feasible solutions satisfying all constraints	Develops workable solutions with minor limitations	Solutions partially satisfy constraints	Solutions do not meet constraints
Application of Concepts	15%	Effectively applies appropriate concepts, methods, or tools	Applies concepts with minor errors	Limited application; requires guidance	Unable to apply relevant concepts
Justification	10%	Provides strong justification for decisions with logical reasoning	Provides justification with some clarity	Weak or unclear justification	No justification provided

SIMATS ENGINEERING

Assessment Tool 2: Annexure A, Constraint-Based Problem Solving

Course: Artificial Intelligence (CSA17) CO1 (BL3) Marks: 30

Name / Reg No: R.Nivetha/ 1912260554

Signature: Nivetha

Question 1: Doctor shift scheduling (Backtracking)

This is a CSP. Variables are the doctors D1, D2, D3. The domain of each is {Morning (M), Afternoon (A), Night (N)}. Every doctor gets exactly one shift and every shift gets exactly one doctor, so the assignment is one-to-one.

Constraint analysis

- (i) D1 ≠ N and (iv) D3 ≠ M are unary constraints. I apply them first, so the domains shrink before any search starts: D1 = {M, A}, D3 = {A, N}, D2 = {M, A, N}.
- (ii) D2 before D3 means D2's shift must come earlier in the order M < A < N than D3's.
- (iii) and (v) together make this an all-different constraint, so no two doctors share a shift.

Backtracking search (order D1, D2, D3; values tried M, A, N)

Step	Assignment tried	Constraint check	Result
1	D1 = M	(i) M is not Night, so it holds. (iii) nothing else assigned yet	Accept
2	D2 = M	(iii) Morning is already taken by D1	Fail, try next value
3	D2 = A	(iii) Afternoon is free. (ii) cannot be checked until D3 is set	Accept
4	D3 = M	(iv) D3 cannot work Morning. (iii) Morning is also taken	Fail, try next value
5	D3 = A	(iii) Afternoon is already taken by D2	Fail, try next value
6	D3 = N	(iii) free. (iv) holds. (ii) D2 = A comes before D3 = N, holds. (v) all three assigned	Accept, goal reached

Final valid schedule: D1 = Morning, D2 = Afternoon, D3 = Night. All five constraints are satisfied.

Justification

Backtracking suits a CSP because it checks constraints after every single assignment and drops a bad branch immediately instead of building a full schedule first. Here it needed only two backtracks. Note that D1 = A, D2 = M, D3 = N is also valid, but the search stops at the first solution it finds with this value order.

Question 2: Robot in a 5×5 grid (BFS with Manhattan heuristic)

Problem understanding and assumed grid

The question does not include the grid figure, so I assume the layout below. Cells are written (row, column) from (0,0). S = (0,0), G = (4,4), obstacles X = (1,1), (2,3), (3,1).

S				
	X			
			X	
	X			
				G

Constraint analysis

Moves are Up, Down, Left, Right at cost 1 each, and obstacle cells cannot be entered. The heuristic is h(n) = |row − 4| + |col − 4|. This never overestimates because the robot cannot move diagonally, so every step reduces the distance by at most 1. The heuristic is admissible and consistent.

Method

Plain BFS expands level by level and would not use a priority queue or a heuristic. As the question asks for both, I keep the BFS frontier but order it by f(n) = g(n) + h(n), lowest first. Ties go to the lower h, then to the order Up, Down, Left, Right. Each entry below is written as cell [g, h, f].

Step	Expand	New nodes added	Priority queue after step	Note
1	(0,0)	(1,0) [1,7,8], (0,1) [1,7,8]	(1,0), (0,1)	Start
2	(1,0)	(2,0) [2,6,8]. (1,1) is an obstacle	(2,0), (0,1)	
3	(2,0)	(3,0) [3,5,8], (2,1) [3,5,8]	(3,0), (2,1), (0,1)	
4	(3,0)	(4,0) [4,4,8]. (3,1) is an obstacle	(4,0), (2,1), (0,1)	
5	(4,0)	(4,1) [5,3,8]	(4,1), (2,1), (0,1)	
6	(4,1)	(4,2) [6,2,8]. (3,1) is an obstacle	(4,2), (2,1), (0,1)	
7	(4,2)	(3,2) [7,3,10], (4,3) [7,1,8]	(4,3), (2,1), (0,1), (3,2)	(3,2) has f = 10
8	(4,3)	(3,3) [8,2,10], (4,4) [8,0,8]	(4,4), (2,1), (0,1), (3,2), (3,3)	
9	(4,4)	none	goal popped	Goal reached

Optimal path: (0,0) → (1,0) → (2,0) → (3,0) → (4,0) → (4,1) → (4,2) → (4,3) → (4,4). **Cost = 8 moves = 8.**

Justification

The cost equals the Manhattan distance from S to G, which is the lowest any path can have, so 8 is optimal. Because all moves cost 1, ordinary BFS would also return a cost of 8, but it would expand many more cells. Using f(n) kept the search on the f = 8 route and only 9 nodes were expanded.

Question 3: Autonomous rescue robot (partially observable, online search)

a) Effect of the constraints on decision-making

Constraint	How it affects the robot
Moves: up, down, left, right	Branching factor is at most 4, and the state space is a small discrete grid.
Blocked cells	Some actions become illegal. The robot learns this only when the cell is sensed, so the plan can break midway.
Sensing only adjacent cells	The environment is partially observable. The robot cannot plan the full route in advance and must treat unseen cells as unknown.
Cost 1 per move, +2 for risky zones	Entering a risky cell costs 3, so a safe detour of up to 2 extra steps is as good as or better than crossing one risky cell. Costs are non-uniform, so plain BFS is not enough.
Minimise cost with safe navigation	Cost is the objective, and safety is a strong preference. When two paths tie on cost, the robot chooses the one without risky cells.
Update knowledge dynamically	The robot interleaves planning and acting. It keeps a map, adds each new observation, and replans when the map changes.

Priority order: blocked cells (hard constraint) first, then total cost, then risk as the tie-breaker, with sensing limits shaping how often the robot replans.

b) Solution approach: online Uniform Cost Search with replanning

- Keep a knowledge map. Every cell starts as unknown and is treated as free with cost 1. This optimistic assumption keeps the plan usable.

- Sense the four neighbours at the current cell and record blocked or risky cells. Entering a risky cell costs 1 + 2 = 3.
- Run Uniform Cost Search on the current map, from the robot's cell to G. It expands the lowest g(n) first, which is correct here because step costs vary. Ties are broken in favour of paths with no risky cell.
- Take only the first step of the best plan, then sense again.
- If the new observation blocks the plan or raises its cost, discard it and rerun step 3 from the current cell. Repeat until G is reached.

Worked example (assumed 5×5 map)

Blocked: (1,2) and (3,2). Risky: (0,3). S = (0,0), G = (4,4). The robot does not know this in advance. With ties on cost, it tries moving Right first.

Robot at	Newly sensed	Decision
(0,0)	(0,1), (1,0) free	Best known plan costs 8. Move Right to (0,1).
(0,1)	(1,1), (0,2) free	Plan still costs 8. Move Right to (0,2).
(0,2)	(1,2) blocked, (0,3) risky	Replan. Via (0,3) costs 3+1+1+1+1+1 = 8 but is risky. Back via (0,1), (1,1), (2,1), (3,1), (4,1), (4,2), (4,3), (4,4) also costs 8 and is safe. Take the safe one.
(0,1), (1,1), (2,1)	(3,1) free, no new blocks	Keep going down the column.
(3,1)	(3,2) blocked, (4,1) free	Plan unaffected. Move to (4,1), then right along the bottom row to G.

Total cost = 10 moves × 1 = 10 with no risky cell entered. An agent that knew the whole map would pay 8, so the extra 2 is the price of having to explore with limited sensing.

c) AI concepts applied

- **Intelligent agent:** The robot perceives (adjacent cells), acts (four moves) and keeps an internal map, so it is a model-based, goal-based agent that also weighs utility (cost and risk). PEAS: Performance is lowest cost with safe arrival. Environment is the damaged building grid. Actuators are the movement motors. Sensors detect adjacent cells.
- **Rationality:** A rational agent picks the action expected to maximise performance given its percept history and knowledge. Here that means choosing the lowest-cost safe step on the current map and replanning when new percepts arrive. Rationality does not mean knowing everything. Going back from (0,2) was rational because it was the best choice for what the robot knew.
- **Environment type:** Partially observable, single agent, deterministic (a move goes where intended), sequential (each move affects later cost), static while the robot plans, discrete, and initially unknown. These properties are why online search is needed instead of offline planning.

d) Why this approach is the most suitable

- Costs are not uniform because of risky zones, so BFS would ignore risk and may return a dangerous route. UCS is complete and optimal for non-negative step costs.
- The map is unknown beyond adjacent cells, so an offline plan could not be built. Online search with replanning handles the surprises.
- Greedy or A* style methods need a goal-distance estimate, which would require knowing the layout. UCS needs only the known path costs, which also makes it safer here.
- It is cheap to run. The state space is a small grid, so replanning after each observation costs little.
- Its tie-breaking rule directly encodes the safety requirement.

SIMATS ENGINEERING

Assessment Tool 2: Annexure A, Constraint-Based Problem Solving

Course: Artificial Intelligence (CSA17) CO1 (BL3) Marks: 30

Name / Reg No: R.Nivetha/ 1912260554

Signature: Nivetha

Question 1: Doctor shift scheduling (Backtracking)

This is a CSP. Variables are the doctors D1, D2, D3. The domain of each is {Morning (M), Afternoon (A), Night (N)}. Every doctor gets exactly one shift and every shift gets exactly one doctor, so the assignment is one-to-one.

Constraint analysis

- (i) D1 ≠ N and (iv) D3 ≠ M are unary constraints. I apply them first, so the domains shrink before any search starts: D1 = {M, A}, D3 = {A, N}, D2 = {M, A, N}.
- (ii) D2 before D3 means D2's shift must come earlier in the order M < A < N than D3's.
- (iii) and (v) together make this an all-different constraint, so no two doctors share a shift.

Backtracking search (order D1, D2, D3; values tried M, A, N)

Step	Assignment tried	Constraint check	Result
1	D1 = M	(i) M is not Night, so it holds. (iii) nothing else assigned yet	Accept
2	D2 = M	(iii) Morning is already taken by D1	Fail, try next value
3	D2 = A	(iii) Afternoon is free. (ii) cannot be checked until D3 is set	Accept
4	D3 = M	(iv) D3 cannot work Morning. (iii) Morning is also taken	Fail, try next value
5	D3 = A	(iii) Afternoon is already taken by D2	Fail, try next value
6	D3 = N	(iii) free. (iv) holds. (ii) D2 = A comes before D3 = N, holds. (v) all three assigned	Accept, goal reached

Final valid schedule: D1 = Morning, D2 = Afternoon, D3 = Night. All five constraints are satisfied.

Justification

Backtracking suits a CSP because it checks constraints after every single assignment and drops a bad branch immediately instead of building a full schedule first. Here it needed only two backtracks. Note that D1 = A, D2 = M, D3 = N is also valid, but the search stops at the first solution it finds with this value order.

Question 2: Robot in a 5×5 grid (BFS with Manhattan heuristic)

Problem understanding and assumed grid

The question does not include the grid figure, so I assume the layout below. Cells are written (row, column) from (0,0). S = (0,0), G = (4,4), obstacles X = (1,1), (2,3), (3,1).

S				
	X			
			X	
	X			
				G

Constraint analysis

Moves are Up, Down, Left, Right at cost 1 each, and obstacle cells cannot be entered. The heuristic is h(n) = |row − 4| + |col − 4|. This never overestimates because the robot cannot move diagonally, so every step reduces the distance by at most 1. The heuristic is admissible and consistent.

Method

Plain BFS expands level by level and would not use a priority queue or a heuristic. As the question asks for both, I keep the BFS frontier but order it by f(n) = g(n) + h(n), lowest first. Ties go to the lower h, then to the order Up, Down, Left, Right. Each entry below is written as cell [g, h, f].

Step	Expand	New nodes added	Priority queue after step	Note
1	(0,0)	(1,0) [1,7,8], (0,1) [1,7,8]	(1,0), (0,1)	Start
2	(1,0)	(2,0) [2,6,8]. (1,1) is an obstacle	(2,0), (0,1)	
3	(2,0)	(3,0) [3,5,8], (2,1) [3,5,8]	(3,0), (2,1), (0,1)	
4	(3,0)	(4,0) [4,4,8]. (3,1) is an obstacle	(4,0), (2,1), (0,1)	
5	(4,0)	(4,1) [5,3,8]	(4,1), (2,1), (0,1)	
6	(4,1)	(4,2) [6,2,8]. (3,1) is an obstacle	(4,2), (2,1), (0,1)	
7	(4,2)	(3,2) [7,3,10], (4,3) [7,1,8]	(4,3), (2,1), (0,1), (3,2)	(3,2) has f = 10
8	(4,3)	(3,3) [8,2,10], (4,4) [8,0,8]	(4,4), (2,1), (0,1), (3,2), (3,3)	
9	(4,4)	none	goal popped	Goal reached

Optimal path: (0,0) → (1,0) → (2,0) → (3,0) → (4,0) → (4,1) → (4,2) → (4,3) → (4,4). **Cost = 8 moves = 8.**

Justification

The cost equals the Manhattan distance from S to G, which is the lowest any path can have, so 8 is optimal. Because all moves cost 1, ordinary BFS would also return a cost of 8, but it would expand many more cells. Using f(n) kept the search on the f = 8 route and only 9 nodes were expanded.

Question 3: Autonomous rescue robot (partially observable, online search)

a) Effect of the constraints on decision-making

Constraint	How it affects the robot
Moves: up, down, left, right	Branching factor is at most 4, and the state space is a small discrete grid.
Blocked cells	Some actions become illegal. The robot learns this only when the cell is sensed, so the plan can break midway.
Sensing only adjacent cells	The environment is partially observable. The robot cannot plan the full route in advance and must treat unseen cells as unknown.
Cost 1 per move, +2 for risky zones	Entering a risky cell costs 3, so a safe detour of up to 2 extra steps is as good as or better than crossing one risky cell. Costs are non-uniform, so plain BFS is not enough.
Minimise cost with safe navigation	Cost is the objective, and safety is a strong preference. When two paths tie on cost, the robot chooses the one without risky cells.
Update knowledge dynamically	The robot interleaves planning and acting. It keeps a map, adds each new observation, and replans when the map changes.

Priority order: blocked cells (hard constraint) first, then total cost, then risk as the tie-breaker, with sensing limits shaping how often the robot replans.

b) Solution approach: online Uniform Cost Search with replanning

- Keep a knowledge map. Every cell starts as unknown and is treated as free with cost 1. This optimistic assumption keeps the plan usable.

- Sense the four neighbours at the current cell and record blocked or risky cells. Entering a risky cell costs 1 + 2 = 3.
- Run Uniform Cost Search on the current map, from the robot's cell to G. It expands the lowest g(n) first, which is correct here because step costs vary. Ties are broken in favour of paths with no risky cell.
- Take only the first step of the best plan, then sense again.
- If the new observation blocks the plan or raises its cost, discard it and rerun step 3 from the current cell. Repeat until G is reached.

Worked example (assumed 5×5 map)

Blocked: (1,2) and (3,2). Risky: (0,3). S = (0,0), G = (4,4). The robot does not know this in advance. With ties on cost, it tries moving Right first.

Robot at	Newly sensed	Decision
(0,0)	(0,1), (1,0) free	Best known plan costs 8. Move Right to (0,1).
(0,1)	(1,1), (0,2) free	Plan still costs 8. Move Right to (0,2).
(0,2)	(1,2) blocked, (0,3) risky	Replan. Via (0,3) costs 3+1+1+1+1+1 = 8 but is risky. Back via (0,1), (1,1), (2,1), (3,1), (4,1), (4,2), (4,3), (4,4) also costs 8 and is safe. Take the safe one.
(0,1), (1,1), (2,1)	(3,1) free, no new blocks	Keep going down the column.
(3,1)	(3,2) blocked, (4,1) free	Plan unaffected. Move to (4,1), then right along the bottom row to G.

Total cost = 10 moves × 1 = 10 with no risky cell entered. An agent that knew the whole map would pay 8, so the extra 2 is the price of having to explore with limited sensing.

c) AI concepts applied

- **Intelligent agent:** The robot perceives (adjacent cells), acts (four moves) and keeps an internal map, so it is a model-based, goal-based agent that also weighs utility (cost and risk). PEAS: Performance is lowest cost with safe arrival. Environment is the damaged building grid. Actuators are the movement motors. Sensors detect adjacent cells.
- **Rationality:** A rational agent picks the action expected to maximise performance given its percept history and knowledge. Here that means choosing the lowest-cost safe step on the current map and replanning when new percepts arrive. Rationality does not mean knowing everything. Going back from (0,2) was rational because it was the best choice for what the robot knew.
- **Environment type:** Partially observable, single agent, deterministic (a move goes where intended), sequential (each move affects later cost), static while the robot plans, discrete, and initially unknown. These properties are why online search is needed instead of offline planning.

d) Why this approach is the most suitable

- Costs are not uniform because of risky zones, so BFS would ignore risk and may return a dangerous route. UCS is complete and optimal for non-negative step costs.
- The map is unknown beyond adjacent cells, so an offline plan could not be built. Online search with replanning handles the surprises.
- Greedy or A* style methods need a goal-distance estimate, which would require knowing the layout. UCS needs only the known path costs, which also makes it safer here.
- It is cheap to run. The state space is a small grid, so replanning after each observation costs little.
- Its tie-breaking rule directly encodes the safety requirement.